Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 9

Team No.: 13

Project Title: System Call Monitor

Team Members

Roll Number	Student Name	Signature
2520030499	G . Sahithi	
2520030557	B . Jasmitha	

1. Abstract

The **System Call Monitor** is a Linux-based C programming project developed to provide a practical understanding of how user-space applications communicate with the operating system kernel through system calls. The system demonstrates essential file and process management operations using system calls such as `open()`, `read()`, `write()`, `close()`, `fork()`, `exec()`, and `wait()`/`waitpid()`.

The project creates and manipulates files, creates and manages processes, executes external programs, and synchronizes parent and child processes. It also implements proper system-call return-value checking and uses `errno` to identify and handle runtime errors. The `strace` utility is integrated to trace the system calls generated during program execution, allowing users to observe the sequence of interactions between an application and the Linux kernel.

By combining implementation with system-call tracing and monitoring, the project provides a hands-on demonstration of **user-space to kernel-space interaction, process management, file operations, error handling, and operating-system services**. It is intended as an educational tool for students to understand the practical working of Linux system calls beyond theoretical concepts.

2 . Problem Statement

Understanding how applications interact with an operating system is a fundamental concept in systems programming and operating systems. Applications cannot directly access most system resources; instead, they communicate with the Linux kernel through **system calls** to perform operations such as creating and accessing files, managing processes, executing programs, and synchronizing tasks.

However, students often learn these concepts primarily through theory, making it difficult to visualize what happens between a user-space application and the kernel. The sequence of system calls, their return values, error conditions, and the transition from user space to kernel space are not always easy to observe during normal program execution.

To address this gap, there is a need for a practical system that demonstrates commonly used Linux system calls and allows their execution to be monitored. The proposed **System Call Monitor** provides a C-based environment for performing file operations using `open()`, `read()`, `write()`, and `close()`, along with process management using `fork()`, `exec()`, and `wait()`/`waitpid()`. The system also incorporates return-value validation and `errno` for proper error handling.

Furthermore, the project uses **`strace`** to trace the system calls generated during execution, enabling users to examine how applications request services from the Linux kernel. This provides a practical understanding of system-call execution, process creation and synchronization, file management, error handling, and the interaction between user-space programs and kernel services.

3 . Objectives

main objectives of the **System Call Monitor** are:

- To understand the role and purpose of Linux system calls.
- To demonstrate file operations using `open()`, `read()`, `write()`, and `close()`.
- To understand process creation using `fork()`.
- To demonstrate execution of external programs using `exec()`.
- To implement process synchronization using `wait()` or `waitpid()`.
- To practice checking and handling system-call return values.
- To understand Linux error handling using `errno`.
- To use `strace` for tracing and monitoring system calls.
- To observe the relationship between user-space applications and kernel services.
- To provide practical understanding of file management, process management, synchronization, and system-call execution in Linux.

4 . Proposed Methodology

The proposed system is implemented in **C on a Linux environment** and is divided into modules that demonstrate different categories of system calls.

1. **File Operation Module**
   The program creates or accesses a file using `open()` and performs reading and writing operations using `read()` and `write()`. The file descriptor is released using `close()`.
2. **Process Creation Module**
   The `fork()` system call is used to create a child process. The program displays relevant process information such as PID and PPID to demonstrate the parent-child relationship.
3. **Program Execution Module**
   The child process uses an `exec()` family function to execute another program, demonstrating how a process can replace its existing program image with a new one.
4. **ProcessSynchronizationModule**
   The parent process uses `wait()` or `waitpid()` to wait for the child process to complete, demonstrating process synchronization.

5. **Error Handling Module**
   Every system call is checked for successful execution. When an error occurs, `errno` and appropriate error-reporting functions are used to identify the cause.
6. **System Call Monitoring**
   The completed program is executed using `strace` to capture the system calls generated during execution. The trace output is analyzed to identify file operations, process-related calls, and interactions with kernel services.
7. **User-Kernel Interaction Analysis**
   The collected observations are used to understand how a user-space application requests and receives services from the Linux kernel through system calls.

5 . Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
File Management – `open()`, `read()`, `write()`, `close()`	Creates, opens, reads, writes, and closes files to demonstrate basic file operations through system calls.
Process Creation – `fork()`	Creates a child process from the parent process to demonstrate process creation and parent-child relationships.
Program Execution – `exec()`	Replaces the child process with another program to demonstrate program execution through the `exec()` family of system calls.
Process Identification – `getpid()`, `getppid()`	Retrieves the Process ID (PID) and Parent Process ID (PPID) to identify and monitor processes.
Process Synchronization – `wait()`, `waitpid()`	Allows the parent process to wait for child processes to complete and ensures proper process synchronization.
Error Handling – `errno`	Identifies and reports errors that occur during system-call execution.
System Call Tracing – `strace`	Traces system calls generated by the program and helps observe interactions between the user-space application and Linux kernel.
User Space / Kernel Space	Demonstrates how applications request operating-system services through system calls and transition from user space to kernel services.

6 . Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030499	G . SAHITHI	**File & System Call Module:** Implement file operations using `open()`, `read()`, `write()`, and `close()`. Handle system-call return values and errors using `errno`. Develop and test the core C program.
2520030557	B . JASMITHA	**Process & Monitoring Module:** Implement process creation and execution using `fork()` and `exec()`, synchronize processes using `wait()`/`waitpid()`, and use `strace` to trace and analyze system calls. Document the user-space and kernel-space interaction.

7 . Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Git & GitHub	Used for version control, project collaboration, and maintaining the project repository.
C / C++	Used to implement process creation, signal handling, communication, and process control.
GCC / G++ Compiler	Compiles the C/C++ source code into an executable program.
strace	Used to trace and monitor system calls generated during program execution.

Linux Operating System	Provides the environment for executing and testing Linux system calls.
Linux Terminal / Shell	Used to compile, execute, test, and monitor processes and signals.
Linux System Calls / APIs	Used for file operations, process creation, program execution, and process synchronization.

8 . Expected Outcome

The project is expected to provide a functional C-based Linux system that demonstrates the practical use of system calls for file and process management. It should successfully perform file operations using `open()`, `read()`, `write()`, and `close()`, create and manage processes using `fork()`, execute programs using `exec()`, and synchronize processes using `wait()` or `waitpid()`.

The system should also detect and handle system-call errors using return values and `errno`. Using `strace`, the generated system calls should be traced and analyzed to clearly demonstrate the interaction between the user-space application and Linux kernel services. The project will provide students with a practical understanding of system calls, process management, file operations, error handling, and user-space to kernel-space interaction.

2. Expected Outcome

The project is expected to produce a functional **Linux-based signal communication and process control application** that demonstrates how multiple processes can communicate and coordinate using signals.

1. The program will successfully create **parent and child processes** using `fork()`.
2. Each process will display its **PID and PPID**, allowing processes to be identified and monitored.
3. The parent process will be able to send signals to the child process using `kill()`.
4. The child process will respond appropriately to signals, demonstrating actions such as **pause, resume, and termination**.
5. Signal handlers will demonstrate how processes respond to **asynchronous events** in real time.
6. The program will properly synchronize and terminate processes using mechanisms such as `wait()`.
7. The terminal output will clearly show **process creation, signal transmission, process state changes, and process termination**.
8. Overall, the project will provide practical understanding of **Linux process management, signal-based IPC, synchronization, and system programming**, while demonstrating how signals can be used for efficient process communication and control.

3. GitHub Repository

GitHub Repository Link:

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name:

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature: